

Formation FPGA :

TP n°5 : Domaines d'horloge

Yassine Aouini

Objectifs : 2

Question 1 : 2

Question 2 : 4

Question 3 : 4

Question 4 : 5

Question 5 : 6

Question 6 : 6

Question 7 : 6

Question 8 : 7

Question 9 : 8

Question 10 : 9

Question 11 : 10

Question 12 : 12

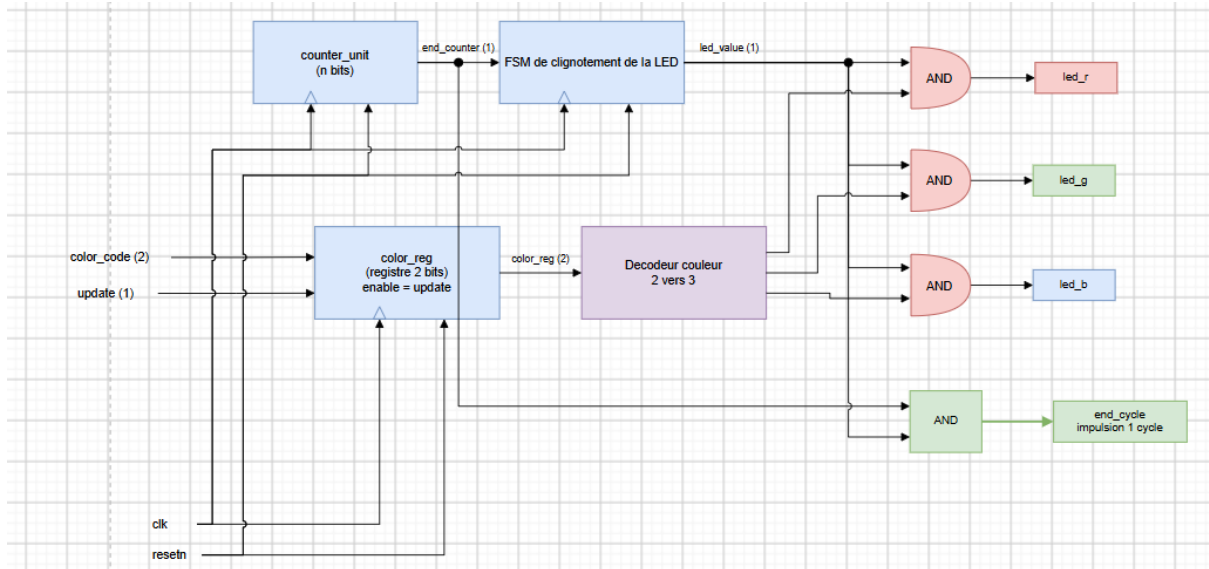
Question 13 : 13

Objectifs :

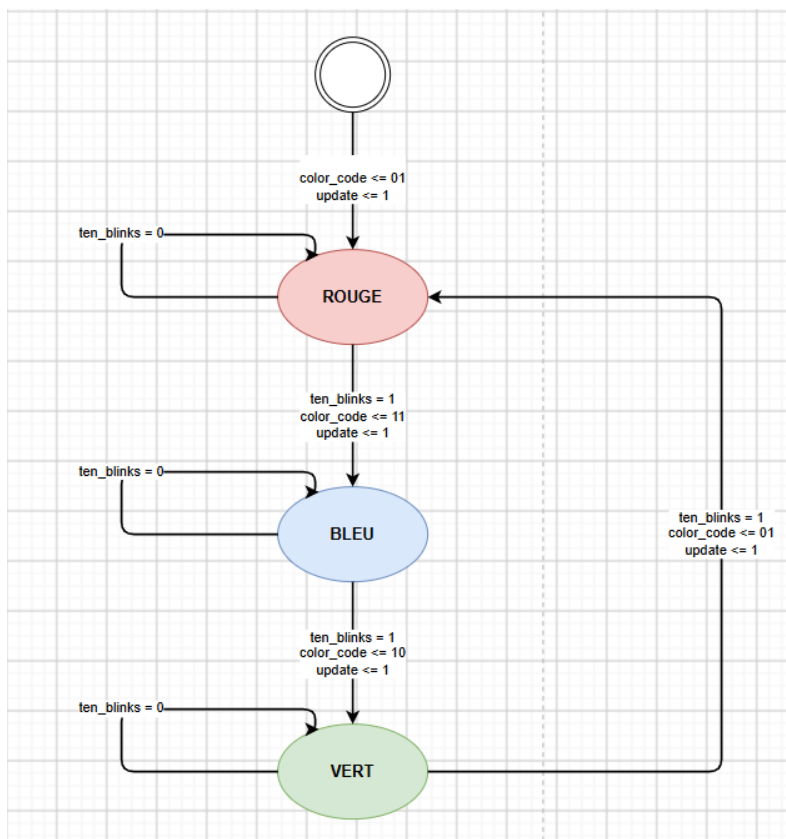
- Mettre en place une architecture utilisant plusieurs domaines d'horloge.
- Faire clignoter deux LED RGB à des fréquences différentes.
- Comprendre et corriger les problèmes liés au franchissement de domaine d'horloge.
- Utiliser une PLL pour générer des horloges.
- Instrumenter un design sur cible à l'aide de sondes ILA.

Question 1 :

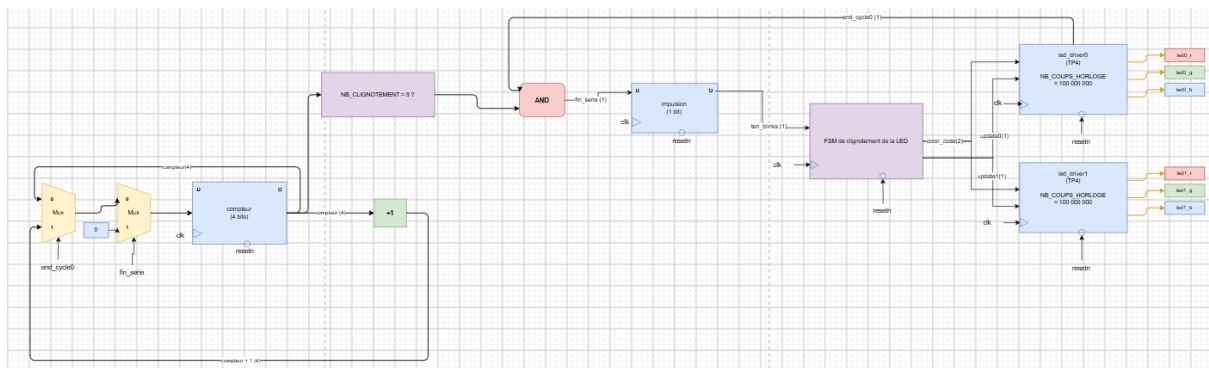
Nous allons réutiliser le module LED_Driver du TP4, avec un changement du nombre de clignotement pour avoir 10 clignotements par couleur affichée.



Voici le schéma RTL de la FSM responsable de la couleur de la LED :



Et voici le schéma complet de l'architecture :



Nom du signal	Nature	Taille en bits
clk_sys	Entrée	1
reset_bt	Entrée	1
led0_r, led0_g, led0_b	Sorties	1 chacune
led1_r, led1_g, led1_b	Sorties	1 chacune
compteur	Signal interne	4
fin_serie	Signal interne	1
ten_blinks	Signal interne, impulsion	1
color_code	Signal interne	2
update	Signal interne, impulsion	1
update0	Signal interne, impulsion	1
update1	Signal interne, impulsion	1
end_cycle0	Signal interne, impulsion	1
end_cycle1	Signal interne, impulsion	1
NB_COUPS_HORLOGE	Générique	100 000 000
NB_CLIGNOTEMENTS	Générique	10

Question 2 : (fichier tp5_top.vhd)

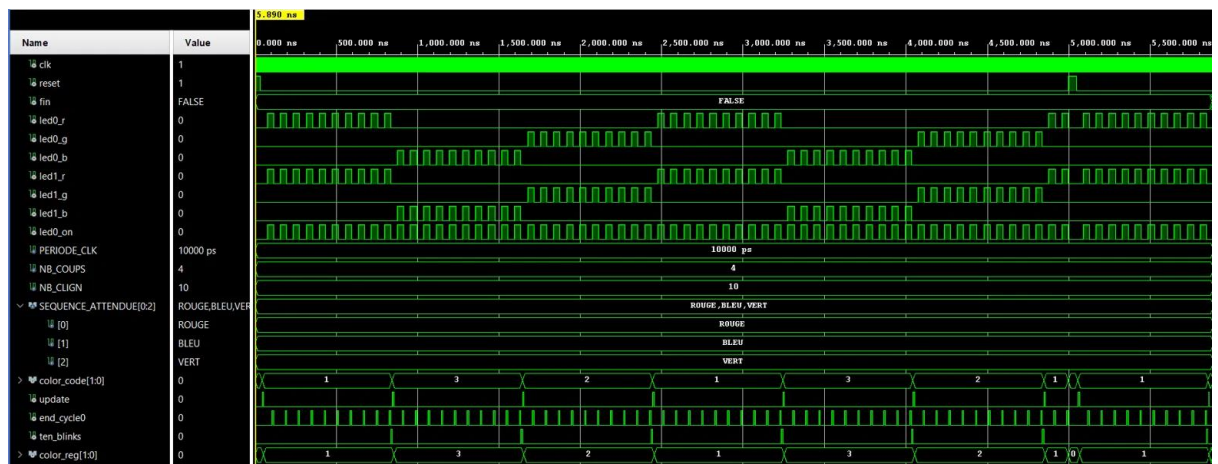
Comme demandé par l'énoncé, update doit être une impulsion.

On souhaite que le signal update et le changement de color_code soient synchronisés. Pour cela, les deux signaux sont affectés au même front d'horloge.

Question 3 : (fichier tp5_top_tb.vhd)

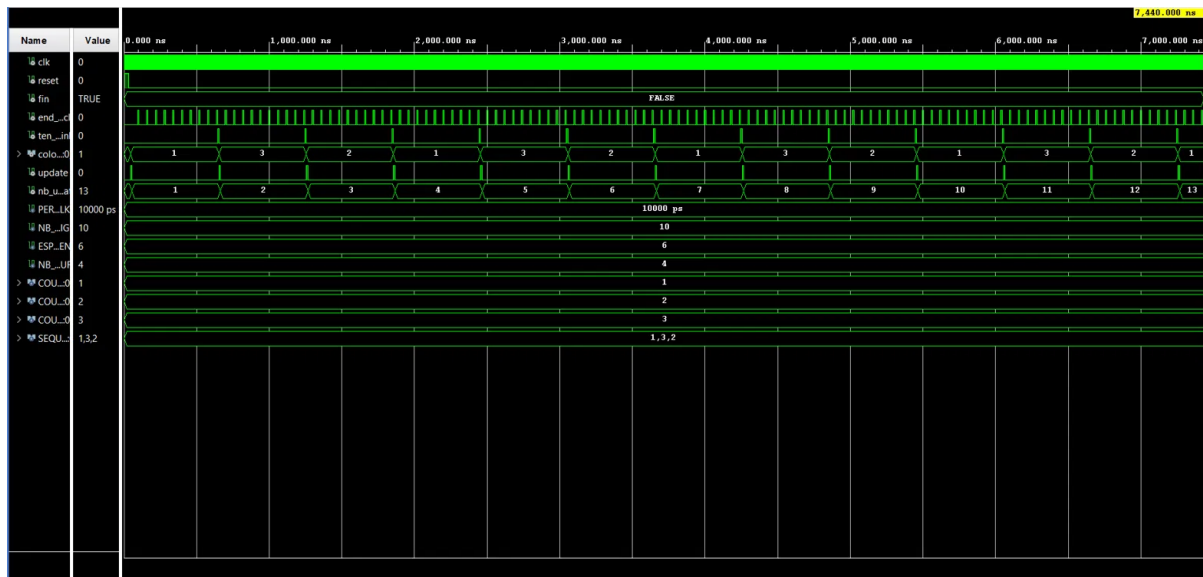
Le testbench permet de simuler et de vérifier les points suivants :

- Les deux LED sont rigoureusement identiques à ce stade du TP, donc leur fonctionnement doit être identique.
- Il n'y a pas de superposition de couleur, chaque couleur est affichée seule.
- Le changement de couleur s'effectue tous les 10 clignotements
- L'ordre des couleurs est respecté, rouge, bleu, vert, puis rouge à nouveau.



On peut observer que les changements de color_reg et color_code sont synchronisés, preuve que les modules réceptionnent correctement le signal update.

Un second testbench permet de s'assurer que le signal update est réellement une impulsion, et que le changement de color_code est bien synchronisé avec le signal update.



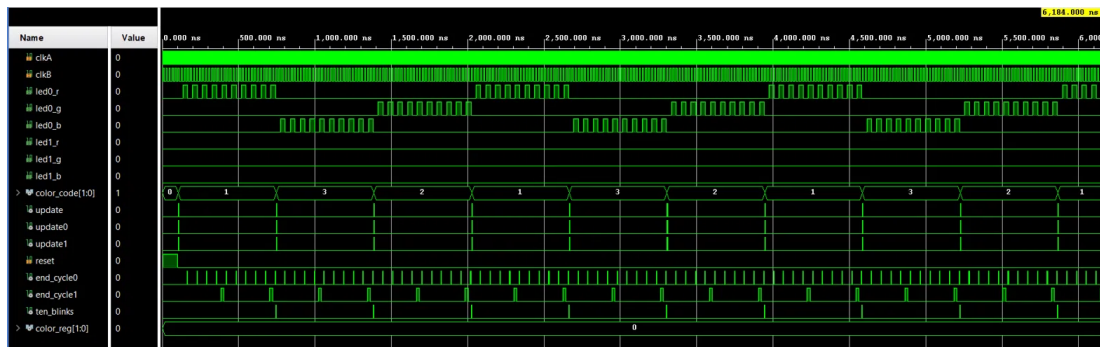
Question 4 : (fichier tp5_top_2clk.vhd)

À l'exception du led_driver1 qui est désormais cadencé sur clkB, l'ensemble de l'architecture est cadencé sur clkA. Aucun changement pour le signal update0, toutefois nous allons étudier le cas du signal update1.

Question 5 : (fichier tp5_top_2clk_tb.vhd)

Les deux horloges sont volontairement déphasées de 3ns afin de bien les différencier. Ainsi, on évite les cas de faux positifs si les fronts montants des deux horloges arrivent simultanément.

Question 6 :



Le premier LED_Driver et sa LED associée fonctionnent comme précédemment, rien d'anormal au niveau du signal update0.

En revanche, un problème apparaît au niveau du signal update1.

Les deux horloges ayant désormais un rapport de 1/5 au niveau de leurs périodes, l'impulsion ne peut être capturée que si le front montant de clkB coïncide avec l'impulsion du signal update1. Cela représente une probabilité de 1/5.

Le décalage de 3ns entre les deux horloges rend cet évènement impossible sur notre simulation.

Toutefois, on remarque que le module LED_Driver1 fonctionne bel et bien. On peut observer sur le chronogramme le bon fonctionnement du signal end_cycle1 ainsi que de l'horloge associée. Le problème ne vient donc pas du module.

Question 7 :

Comme annoncé, le problème est lié au changement de domaine d'horloge du signal update1.

Deux obstacles se présentent :

- La métastabilité, qui survient quand une donnée change pendant la fenêtre de setup et de hold d'une bascule.
- La perte d'évènement, qui survient quand une impulsion disparaît avant qu'un front de capture ne l'ait vue.

D'après le lien fourni par l'énoncé, une solution est un synchroniseur à deux bascules.

C'est la piste que nous allons implémenter puis simuler.

Question 8 :

Après simulation du synchroniseur à deux bascules, on peut voir que le résultat ne diffère pas vraiment de la simulation précédente.

En effet, on n'observe toujours pas de changement de couleur au niveau du color_reg de led_driver1.



Ce montage permet en fait de corriger la métastabilité mais ne permet pas de corriger la perte d'évènement.

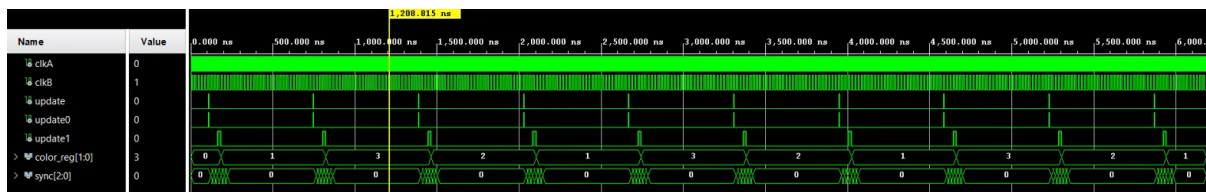
Une autre solution peut être envisagée : l'étirement d'impulsion couplé à une détection de front.

Pour un passage d'un domaine rapide vers un domaine lent, une solution est d'étirer l'impulsion sur au moins deux périodes de l'horloge de destination.

Avec un rapport de 1/5, 2 périodes de clkB équivalent à 10 périodes de clkA. Pour plus de sécurité, nous pouvons prendre une largeur d'impulsion de 16 périodes de clkA.

Une des contraintes de l'exercice est un signal update qui soit une impulsion d'un seul coup d'horloge, ainsi l'étirement ne suffit pas à résoudre le problème.

Nous allons coupler l'étirement avec le synchroniseur précédemment simulé ainsi qu'une détection de front afin de régénérer une impulsion d'un seul coup d'horloge.



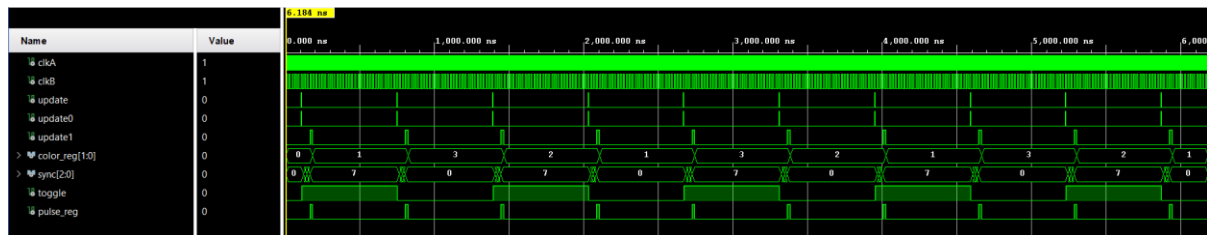
Cette fois, on observe le changement de couleur au niveau de la LED1.

Cette solution présente toutefois un défaut. En effet, elle ne fonctionne que si clkB reste suffisamment rapide par rapport à la largeur d'impulsion choisie.

Une dernière solution permettrait de s'affranchir de cette contrainte : la bascule d'inversion.

Le principe consiste à ne plus faire traverser une impulsion ni une impulsion élargie, mais un changement d'état permanent. Dans le domaine clkA, chaque impulsion inverse une bascule. L'information cesse d'être un événement pour devenir un niveau stable, que le domaine de destination ne peut plus manquer, quelle que soit sa cadence.

Dans le domaine clkB, ce niveau traverse le synchroniseur à deux bascules déjà mis en œuvre, qui absorbe la métastabilité. Une troisième bascule et une porte XOR détectent ensuite le changement de niveau et régénèrent une impulsion d'un seul coup d'horloge.



On observe à nouveau le changement de couleur au niveau de la LED1.

De plus, pour chaque impulsion update du domaine de clkA, il y a une impulsion update1 du domaine de clkB. Cette impulsion update1 reproduit le signal pulse_reg à chaque transition du signal toggle. Le vecteur sync qui représente les trois bascules évoquées dans la solution, passe de 0 à 111 et montre que les bascules prennent la valeur du toggle.

Question 9 : (fichier tp5_top_pll.vhd)

Le nom de l'IP en question est "Clocking Wizard". Deux options sont proposées pour la configuration : "MMCM" et "PLL". Dans notre cas, le choix PLL est le plus simple à utiliser.

On configure le module en y entrant les différentes fréquences souhaitées.

Component Name

clk_wiz_0

Clocking Options

Output Clocks

Port Renaming

PLLE2 Settings

Summary

ase is calculated relative to clk_out1.

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)	
		Requested	Actual	Requested	Actual	Requested	Actual
clk_out1	clk_out1	250.000	250.00000	0.000	0.000	50.000	50.0
clk_out2	clk_out2	50.000	50.00000	0.000	0.000	50.000	50.0
clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A
clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A
clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A
clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A

USE CLOCK SEQUENCING

Clocking Wizard (6.0)

Documentation

IP Location

Switch to Defaults

IP Symbol

Resource

Show disabled ports

+

s_axi_lite

+

CLK_IN1_D

+

CLK_IN2_D

+

CLKFB_IN_D

+

CLKFB_OUT_D

+

s_axi_aclk

+

s_axi_arresetn

+

reset

+

resetn

+

ref_clk

+

user_clk0

+

user_clk1

+

user_clk2

+

user_clk3

+

clk_in1

CLKFB_OUT_D

clk_stop[3:0]

clk_glitch[3:0]

interrupt

clk_oar[3:0]

clk_out1

clk_out2

locked

Component Name

clk_wiz_0

Clocking Options

Output Clocks

Port Renaming

PLLE2 Settings

Summary

Primary Input Clock Attributes

Input Clock Frequency (MHz)	100.000
Clock Source	Single_ended_clock_capable_pin
Jitter	0.010

Clocking Primitive Attributes

Primitive Instantiated : PLL

Divide Counter : 1

Mult Counter : 10

Clock Phase Shift : None

Clock Wiz O/p Pins	Source	Divider Value	Tspread (ps)	Pk-to-Pk Jitter (ps)	Phase Error (ps)
clk_out1	PLL CLKOUT0	4	OFF	110.209	98.575
clk_out2	PLL CLKOUT1	20	OFF	151.636	98.575
clk_out3	OFF	OFF	OFF	OFF	OFF
clk_out4	OFF	OFF	OFF	OFF	OFF
clk_out5	OFF	OFF	OFF	OFF	OFF
clk_out6	OFF	OFF	OFF	OFF	OFF
clk_out7	OFF	OFF	OFF	OFF	OFF

Les deux clock sont bien ajoutées, une à 250MHz et l'autre à 50MHz.

Question 10 :



Set up Debug Summary

- 0 debug cores will be removed
- 2 debug cores will be created
- Found 2 clocks

☒ Open in Debug layout

To apply the above changes, click Finish

Open in the specified layout



< Back

Next >

Finish

Cancel

Nets to Debug

The nets below will be debugged with ILA cores. To add nets click "Find Nets to Add". You can also select nets in the Netlist or other windows, then drag them to the list or click "Add Selected Nets".

Name	Clock Domain	Driver Cell	Probe Type
>  color_code (2)	u_pll/inst/clk_out1	FDCE	Data and Trigger
 end_cycle0	u_pll/inst/clk_out1	LUT2	Data and Trigger
 end_cycle1	u_pll/inst/clk_out2	LUT2	Data and Trigger
 u_pulse_sync/pulse_reg	u_pll/inst/clk_out2	FDCE	Data and Trigger
 ten_blinks	u_pll/inst/clk_out1	FDCE	Data and Trigger
 u_pulse_sync/toggle	u_pll/inst/clk_out1	FDCE	Data and Trigger
 update	u_pll/inst/clk_out1	FDCE	Data and Trigger
 update1	u_pll/inst/clk_out2	FDCE	Data and Trigger

Nous utilisons 2 cores ILA pour étudier les deux horloges du système. En effet, un core ILA ne possède qu'une seule entrée d'horloge, il est donc nécessaire d'en utiliser deux.

Question 11 :

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	88	0	17600	0.50
LUT as Logic	88	0	17600	0.50
LUT as Memory	0	0	6000	0.00
Slice Registers	77	0	35200	0.22
Register as Flip Flop	77	0	35200	0.22
Register as Latch	0	0	35200	0.00
F7 Muxes	0	0	8800	0.00
F8 Muxes	0	0	4400	0.00

Chaque LED_Driver comporte un compteur utilisant 28 bascules, 2 bascules pour color_reg, et une bascule pour led_value soit $2 \times 31 = 62$ bascules.

Le compteur de clignotement utilise 4 bascules pour compter jusqu'à 10 clignotements et un registre pour l'impulsion, soit 5 au total.

La machine à états utilise 2 bascules pour l'état, 2 bascules pour le code couleur et une bascule pour l'update, soit 5 au total.

Le synchroniseur en utilise une pour toggle, trois pour la chaîne de synchronisation et une pour l'impulsion de sortie, soit 5 au total.

On obtient bien les 77 bascules comme indiqué ci-dessus.

Site Type	Used	Fixed	Available	Util%
Slice	1218	0	4400	27.68
SLICEL	782	0		
SLICEM	436	0		
LUT as Logic	2069	0	17600	11.76
using O5 output only	0			
using O6 output only	1733			
using O5 and O6	336			
LUT as Memory	239	0	6000	3.98
LUT as Distributed RAM	24	0		
using O5 output only	0			
using O6 output only	0			
using O5 and O6	24			
LUT as Shift Register	215	0		
using O5 output only	6			
using O6 output only	71			
using O5 and O6	138			
Slice Registers	3647	0	35200	10.36
Register driven from within the Slice	1695			
Register driven from outside the Slice	1952			
LUT in front of the register is unused	1559			
LUT in front of the register is used	393			
Unique Control Sets	193		4400	4.39

En comparaison, les 77 bascules du design seul sont négligeables face aux 3647 bascules totales. La majeure partie des bascules est utilisée par l'ILA afin de stocker l'ensemble des valeurs.

Question 12 :

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0,696 ns	Worst Hold Slack (WHS): 0,029 ns	Worst Pulse Width Slack (WPWS): 1,020 ns
Total Negative Slack (TNS): -10,014 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 28	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 6858	Total Number of Endpoints: 6842	Total Number of Endpoints: 4060
Timing constraints are not met.		

On remarque un problème, les contraintes de temps ne sont pas respectées en setup. Aucune violation de hold n'est relevée.

On étudie chaque domaine pour déterminer d'où vient le problème.

Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)	WPWS (ns)	TPWS (ns)	Failing Endpoints (TPWS)	Total Endpoints (TPWS)
clk_sys											3.000	0.000	0	
clk_out1_clk_wiz_0	rise - rise	-0.696	-10.014	28	403	rise - rise	0.079	0.000	0	403	1.020	0.000	0	
clk_out2_clk_wiz_0	rise - rise	11.411	0.000	0	5294	rise - rise	0.029	0.000	0	5294	8.750	0.000	0	
clkfbout_clk_wiz_0											8.751	0.000	0	
dbg_hub/inst/BSCANID_u_xsc	rise - rise	25.481	0.000	0	933	rise - rise	0.049	0.000	0	933	15.250	0.000	0	

From Clock	To Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)
clk_out1_clk_wiz_0	clk_out2_clk_wiz_0	rise - rise	1.209	0.000	0	3	rise - rise	0.165	0.000	0	3
dbg_hub/inst/BSCANID.u_xsc	clk_out2_clk_wiz_0	rise - rise	31.728	0.000	0	8					
clk_out2_clk_wiz_0	dbg_hub/inst/BSCANID.u_xsc	rise - rise	18.811	0.000	0	8					

La violation se passe dans le domaine de l'horloge clk_out1, dont la fréquence est de 250MHz.

Jusque-là nous avons manipulé des fréquences beaucoup plus basses que 250MHz, donc les ressources étaient surdimensionnées pour notre utilisation.

Regardons en détail le chemin critique de notre design :

```

Max Delay Paths
-----
Slack (VIOLATED) :      -0.696ns (required time - arrival time)
Source:                u_led_driver0/u_counter/count_reg[13]/C
                        (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 (rise@0.000ns fall@2.000ns period=4.000ns))
Destination:           u_led_driver0/u_counter/count_reg[25]/D
                        (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 (rise@0.000ns fall@2.000ns period=4.000ns))
Path Group:            clk_out1_clk_wiz_0
Path Type:             Setup (Max at Slow Process Corner)
Requirement:           4.000ns (clk_out1_clk_wiz_0 rise@4.000ns - clk_out1_clk_wiz_0 rise@0.000ns)
Data Path Delay:       4.549ns (logic 2.311ns (50.802%) route 2.238ns (49.198%))
Logic Levels:          10 (CARRY4=7 LUT2=1 LUT6=2)
Clock Path Skew:       -0.190ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD):  5.831ns = ( 9.831 - 4.000 )
  Source Clock Delay (SCD):        6.391ns
  Clock Pessimism Removal (CPR):   0.370ns
Clock Uncertainty:      0.065ns ((TSJ^2 + DJ^2)^1/2) / 2 + PE
Total System Jitter (TSJ):        0.071ns
Discrete Jitter (DJ):            0.110ns
Phase Error (PE):             0.000ns

```

Le chemin critique est contenu dans le compteur du LED_Driver, on peut voir que le routage est responsable de la moitié du délai de ce chemin.

Les violations observées sont internes au domaine rapide, elles relèvent de la structure du circuit. Ainsi, une solution serait de modifier le compteur interne au LED_Driver.

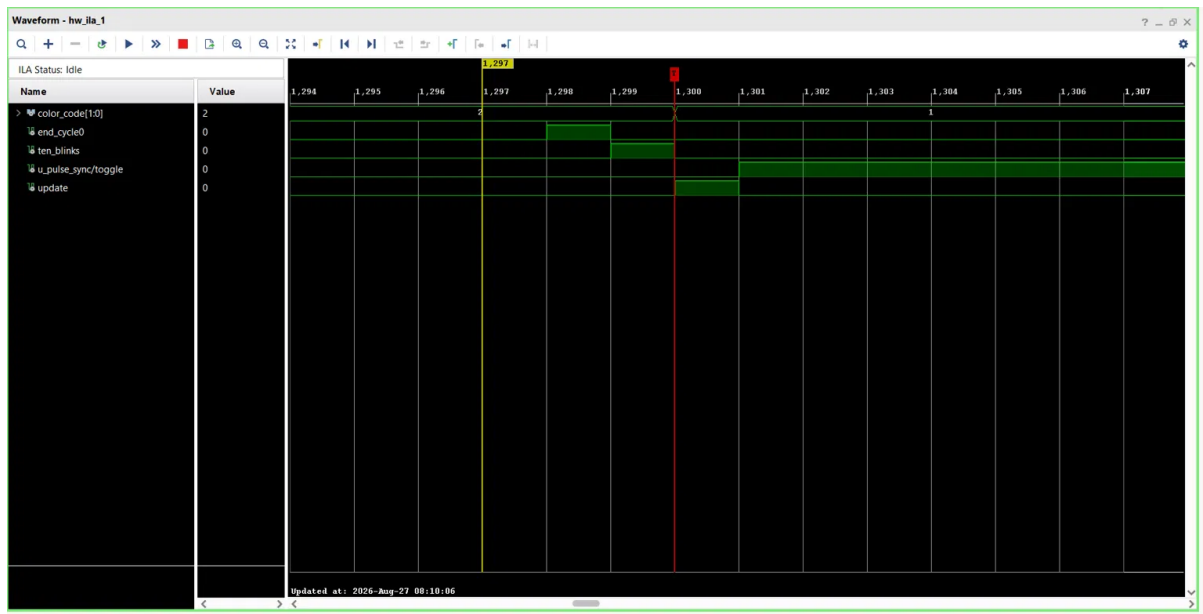
Ceci est contraire à une des règles du TP qui nous demande d'utiliser le même module qu'au TP4.

Question 13 :

Après génération du bitstream et programmation de la carte, on peut vérifier les signaux du chipscope.

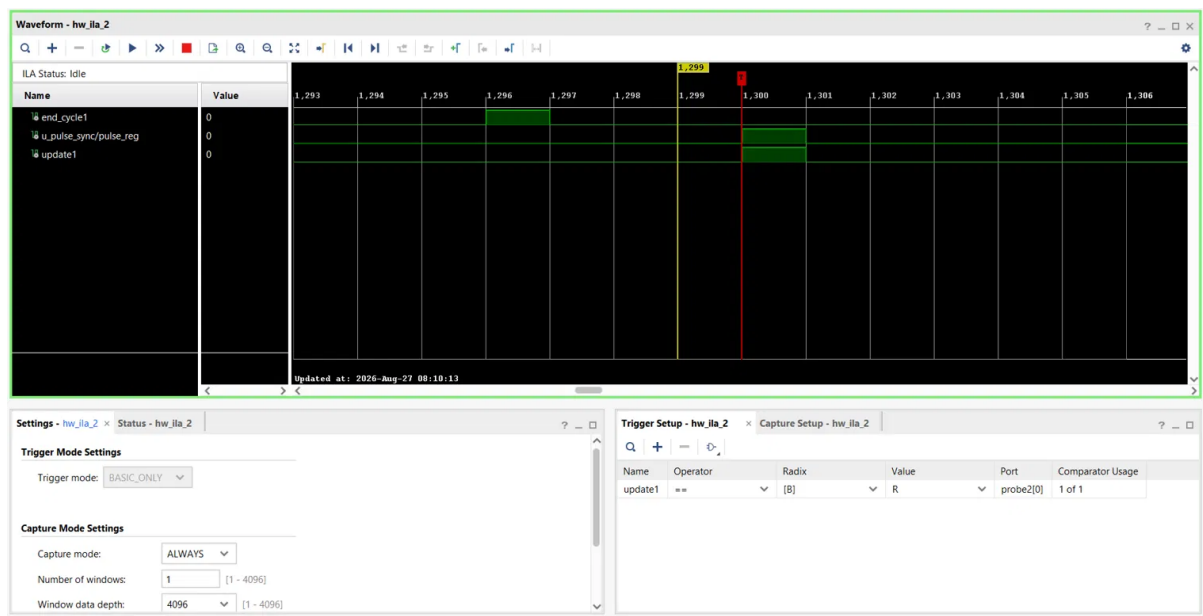
Chaque core a été armé sur une condition de front montant : update pour le core du domaine rapide (clkA) et update1 pour le core du domaine lent (clkB)

Observation du domaine clkA :



On remarque la succession des signaux `end_cycle0`, `ten_blinks` et `update` sur un coup d'horloge chacun. Le signal `update` entraîne le changement de la valeur de `color_code` qui passe de 2 à 1. Le signal `toggle` bascule à 1 pour conserver le niveau.

Observation du domaine clkB :



On peut voir que `pulse_reg` et `update1` présentent une impulsion sur 1 seul coup d'horloge de `clkB`.

Cela montre que le franchissement de domaine a bien régénéré une impulsion dans le domaine de destination.

Observation du comportement sur carte : (cf.vidéo de démonstration)

On observe que pour 5 clignotements de la LED0, on a un clignotement de la LED1.

Toutefois, le changement de couleur des deux LED est identique et boucle indéfiniment.

Un appui sur le bouton reset ramène les LED à l'état initial éteint avant de recommencer la séquence depuis le début.

Bien que le système fonctionne sur carte, la validation du design ne peut pas être faite en raison des violations vues à la question 12.